

1. Cài đặt Docker (ngắn gọn)

- Windows / macOS: cài Docker Desktop từ trang chính thức; bật WSL2 backend trên Windows.
- Linux: cài docker-engine theo distro (apt/yum); thêm user vào nhóm docker: `sudo usermod -aG docker $USER`.
- Kiểm tra: `docker --version` và `docker run --rm hello-world`.

2. Lệnh Docker cơ bản (những lệnh cần nhớ)

- `docker build -t my-image:tag .`
- `docker images` (liệt kê images)
- `docker run -d --name my-container -p 8080:80 my-image:tag`
- `docker ps -a` (liệt kê containers)
- `docker logs -f <container>`
- `docker exec -it <container> /bin/sh` (hoặc `/bin/bash`)
- `docker stop/start/restart <container>`
- `docker rm <container> && docker rmi <image>`
- `docker pull/push <registry>/<repo>:tag`

3. Dockerfile mẫu

Backend Node.js (ví dụ: backend/Dockerfile)

```
# Stage build
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage runtime
FROM node:18-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/dist ./dist
COPY package*.json ./
```

```
RUN npm ci --only=production
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Ghi chú: multi-stage để giảm kích thước image; dùng alpine cho gọn; cài production dependencies.

Frontend React (ví dụ: frontend/Dockerfile)

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:stable-alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4. docker-compose cho dev (local)

Ví dụ docker-compose.yml (root repo) chạy backend, frontend (optional), postgres, redis, và adminer:

```
version: "3.8"
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: bookingdb
      POSTGRES_USER: booking
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

```
backend:
  build:
    context: ./backend
  environment:
    DATABASE_URL: postgres://booking:secret@db:5432/bookingdb
    REDIS_URL: redis://redis:6379
    NODE_ENV: development
  ports:
    - "3000:3000"
  depends_on:
    - db
    - redis

frontend:
  build:
    context: ./frontend
  ports:
    - "5173:80"
  depends_on:
    - backend

volumes:
  db_data:
```

5. Build, tag và push image lên registry

- Đăng nhập registry: `docker login $REGISTRY_URL` (GitLab: `docker login $CI_REGISTRY`).
- Build: `docker build -t $REGISTRY/$GROUP/$PROJECT/backend:$TAG -f backend/Dockerfile backend/`
- Push: `docker push $REGISTRY/$GROUP/$PROJECT/backend:$TAG`
- Kết hợp với GitLab CI: dùng `docker:dind` hoặc `buildx`, lưu credentials trong CI/CD variables (protected + masked).

6. Tích hợp Docker với GitLab CI (tóm tắt)

- Sử dụng job image: `docker:24` + service `docker:dind` hoặc dùng `kaniko/buildkit` để build image không cần privileged runner.
- Lưu Docker credentials vào CI variables (`CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`).

- Ví dụ job: build image → tag với \$CI_COMMIT_SHORT_SHA → push → trigger deploy job (helm/kubectl).

7. Deploy lên Kubernetes (tổng quan)

- Push image lên registry (GitLab Container Registry, ECR, GCR).
- Sử dụng Helm charts hoặc Kubernetes manifests (Deployment, Service, Ingress, ConfigMap, Secret).
- Sử dụng RollingUpdate hoặc Canary (feature flags) để giảm rủi ro.
- Lưu kubeconfig hoặc token an toàn trong CI variables (base64, protected).

8. Secrets, Config và môi trường

- Không commit secrets vào repo. Lưu trong:
 - Kubernetes Secrets (sử dụng sealed-secrets hoặc external Secrets operator để lấy từ Vault), hoặc
 - GitLab CI Variables (protected, masked) cho pipeline.
- Dùng ConfigMap cho cấu hình không nhạy cảm; dùng environment variables để inject vào container.

9. Best practices Dockerfile & images

- Dùng multi-stage builds để giảm size.
- Chỉ include production dependencies cho stage runtime.
- Sử dụng images base nhẹ (alpine) trừ khi cần native libs.
- Thiết lập fixed-version cho base images (node:18-alpine -> node:18.16.0-alpine3.18) để tránh drift.
- Quét image để phát hiện vulnerability (Trivy, Snyk) trong pipeline.
- Thiết lập non-root user trong container khi chạy production.
- Giữ layer caching hợp lý: copy package.json + npm ci trước khi copy toàn bộ source.

10. Bảo mật container

- Scan images bằng Trivy/Clair/Snyk; fail pipeline nếu vulnerability critical.
- Thiết lập runtime hardening: read-only filesystem nếu có thể, drop capabilities, set resource limits.

- Không chạy container dưới root; tạo và chuyển sang non-root user trong Dockerfile.
- Giữ network policy cho Kubernetes, giới hạn truy cập giữa pods.

11. Debugging container & troubleshooting

- Xem logs: `docker logs -f <container>` hoặc `kubectl logs -f deployment/<name>`.
- Vào container: `docker exec -it <container> sh` (hoặc `kubectl exec -it pod -- sh`).
- Kiểm tra health/readiness probes trên K8s nếu pod restart liên tục.
- Nếu build chậm: tối ưu cache, dùng buildKit, tách jobs, tăng runner resources.
- Nếu image quá lớn: kiểm tra Dockerfile layers, remove dev deps, dùng alpine.

12. Clean up & housekeeping

- Xóa containers/images không dùng: `docker system prune -a --volumes` (cẩn thận).
- Trong CI: set `expire_in` cho artifacts và images để tiết kiệm storage.
- Quản lý registry: xóa tags cũ theo retention policy hoặc tự động qua cleanup rules.

13. Checklist triển khai Docker cho dự án Booking Hotel

1. Viết Dockerfile cho backend và frontend (multi-stage).
2. Tạo docker-compose.yml cho dev (db, redis, backend, frontend).
3. Cấu hình CI job build/push image; lưu credentials an toàn.
4. Tạo Helm chart / K8s manifests cho deploy staging & production.
5. Thiết lập image scan (Trivy) trong pipeline.
6. Thiết lập runtime security rules và resource limits.
7. Tạo docs README: cách build/run local, debug, và rollback steps.